
Bayesian Inference in Diffusion Tensor Imaging

Abhinav Ramalingam

Masters in Computer Science
Uppsala University, Sweden

abhinav.ramalingam.2263@student.uu.se

Hampus Arnestrand

Engineering Physics Programme
Uppsala University, Sweden

hampus.arnestrand.6006@student.uu.se

Ibtiaj Mahmud Diljar

Masters in Computer Science
Uppsala University, Sweden

ibtiaj-mahmud.diljar.7719@student.uu.se

Torin Wolke

Engineering Physics Programme
Uppsala University, Sweden

torin.wolke.7786@student.uu.se

Abstract

This project applies Bayesian inference to the diffusion tensor model (DTI) to infer the posterior distribution of the baseline signal (S_0) and the diffusion tensor (D) for a single voxel using real-world diffusion MRI data. We implemented and rigorously compared four distinct approximate inference techniques: Metropolis-Hastings (MH), Importance Sampling (IS), Variational Inference (VI), and the Laplace Approximation (LA). The underlying Bayesian model incorporated Gamma priors on S_0 and the tensor eigenvalues, requiring the optimization methods (VI and LA) to utilize reparameterization based on Cholesky factorization and log-transformation to ensure physical constraints such as positivity and positive definiteness were met. The stochastic sampling techniques, Metropolis-Hastings (MH) and Importance Sampling (IS), which asymptotically converge to the true posterior, together with the deterministic Laplace Approximation (LA), yielded highly accurate and consistent posterior estimates (e.g., S_0 mean $\approx 335 - 338$ and similar Mean Diffusivity/Fractional Anisotropy values), confirming they converged to the same posterior mode. In comparing the methods for scalability, particularly concerning a full-brain analysis of $\approx 10^5$ voxels, the deterministic approaches—LA and VI—were determined to be the most practical solutions due to their speed and inherent voxel-wise parallelizability, offering a better balance between computational cost and approximation quality than the resource-intensive and sequential sampling techniques.

Author Contribution: Abhinav Ramalingam: Variational Inference, writing - Abstract, Introduction. Hampus Arnestrand: Metropolis-Hastings and helped write the introduction. Ibtiaj Mahmud Diljar: Laplace Approximation, Posterior Metrics, writing – Section 1.2 (Likelihood), Discussion. Torin Wolke: Implemented importance sampling and the logpdf method of the prior. Wrote sections 1.1 (Prior) and 2.2 (Importance sampling). All Members: writing - Their own method in Report.

1 Introduction

Diffusion MRI non-invasively measures the movement of water molecules in tissues, revealing the microstructure of white matter. Key parameters are the diffusion tensor D , describing directional diffusion, and the baseline signal S_0 , representing the signal without diffusion weighting. Using a Bayesian approach, we infer the posterior distribution of (S_0, D) , capturing both estimates and uncertainty. This project aims to estimate S_0 and D for a single voxel from observed diffusion MRI data using Metropolis-Hastings, Importance Sampling, Variational Inference, and Laplace Approximation.

We get some diffusion-weighted MRI data from the Stanford HARDI dataset. The code preprocess this data by masking and cropping away the background until we are left with a single voxel and the corresponding diffusion tensor model (DTI). This is used to extract a point estimate used as a starting point for the implemented inference methods. The point estimate includes the baseline signal S_0 , and the eigenvalues and eigenvectors of the diffusion tensor D for the chosen voxel. We also use the voxels observed MRI signal vector and gradient table to define the likelihood.

1.1 Prior

The prior distributions that were used for the latent variables are given by:

$$S_0 \sim \text{Gamma}(\alpha_S, \theta_S) \quad (1)$$

$$\lambda_1, \lambda_2, \lambda_3 \sim \text{Gamma}(\alpha_\lambda, \theta_\lambda) \quad (2)$$

$$\mathbf{V} \sim \text{Uniform}(\text{SO}(3)) \quad (3)$$

with

$$\text{Gamma}(\alpha, \theta) = \frac{1}{\Gamma(\alpha)\theta^\alpha} x^{\alpha-1} e^{-x/\theta}. \quad (4)$$

and $\alpha_S = 2$, $\theta_S = 500$, $\alpha_\lambda = 4$, $\theta_\lambda = 2.5 \cdot 10^{-4}$.

During the entire project, log-probabilities were used to avoid numerical underflow. In our implementation of the prior, we therefore calculated the logarithm of the probability for each latent variable and then summed the log-probabilities to get the total log-probability of the prior.

1.2 Likelihood

For our likelihood implementation, we modeled the relationship between the observed diffusion MRI signals and the underlying tissue parameters. We assumed that each measurement follows the standard diffusion tensor imaging signal equation, where the signal S_i equals the baseline signal S_0 multiplied by the exponential of the negative quadratic form involving the diffusion tensor D and the gradient direction q_i . To account for measurement noise, we incorporated a Gaussian noise model in the signal domain with a fixed standard deviation of 29, which is typical for diffusion MRI data. The likelihood function computes the probability of observing our actual voxel measurements given any proposed set of parameters, forming the crucial data fidelity term in our Bayesian inference framework.

1.3 MVN Reparameterized

We developed the `mvn_reparameterized` class to represent our Laplace approximation as a multivariate normal distribution while respecting the natural constraints of our parameters. Since the baseline signal S_0 must be positive and the diffusion tensor D must be positive definite, we could not simply use a standard multivariate normal distribution on the original parameters. Instead, we worked in a transformed space where we used the logarithm of S_0 and the six-dimensional θ parameterization of D derived from its Cholesky factorization. This reparameterization ensures that when we sample from our multivariate normal approximation and transform back to the original space, we automatically satisfy all necessary constraints without requiring rejection sampling or truncation.

1.4 Metrics

The `plot_results` function is used to visualize the approximated baseline signal, mean diffusivity, fractional anisotropy, and acute angle, as distributions through the use of histograms.

2 Bayesian Approximation Methods

2.1 The Metropolis-Hastings algorithm

The Metropolis-Hastings algorithm is a Markov chain that works by drawing proposal samples to approximate the posterior. The samples are either accepted or rejected based on a probability evaluation using the prior and likelihood with equation 5. The new value is accepted with probability a . When rejected the previous state is replicated.

$$a = \frac{\pi(x') q(x^{(t-1)} | x')}{\pi(x^{(t-1)}) q(x' | x^{(t-1)})}. \quad (5)$$

where $\pi(x) = p(y|x)p(x)$.

Normally the proposal can be Gaussian, but in this case we have some specific constraints and dependencies that make the Gaussian a poor choice. The diffusion signal S has to be nonnegative, while the Gaussian can produce negative values. The diffusion tensor D is symmetric, and with correlated values that make it unlikely to get accepted if proposed by a simple diagonal Gaussian distribution.

So, for the proposal of $\mathbf{z}'$ we use equations 6, 7, and 8.

$$q(\mathbf{z}|\mathbf{z}') = q(S'_0|S_0)q(\mathbf{D}'|\mathbf{D}) \quad (6)$$

$$q(S'_0|S_0) = \text{Gamma}(\gamma^{-2}, \gamma^2 S_0) \quad (7)$$

$$q(\mathbf{D}'|\mathbf{D}) = W(\mathbf{D}, \nu) \quad (8)$$

The mean of the gamma distribution in equation 7 is the shape parameter times the scale

$$\mathbb{E}[S'_0] = \gamma^{-2} \gamma^2 S_0 = S_0$$

and the variance is

$$\text{Var}(S'_0) = \gamma^{-2} (\gamma^2 S_0)^2 = \gamma^2 S_0^2$$

So we know that the variance of the S_0 sampling scales quickly with γ .

In the scipy implementation of the Wishart distribution (equation 8) the mean is νD . So we set $scale = D_{prev}/\nu$ to achieve sampling around the previous value. Resulting in a variance that scales with $1/\nu$ for D .

To find γ and ν that resulted in an acceptance rate of around 50%, a simple grid search was conducted. As could have been guessed, we get a high acceptance rate with small variance (small γ and large ν), as we start with the point estimate from the data and always propose very close values. However, we also need some exploration to cover the whole posterior distribution with a limited number of samples. A good balance, resulting in around 50% acceptance and high enough variances for both distributions, was found with $\gamma = 0.01$ and $\nu = 2000$. Figure 1 shows the resulting posterior distributions using histograms of the 10000 sampled values.

Trace plots of S_0 , D s eigenvalues and one eigenvector component showed that we had no apparent burn-in period when starting the sampling from the data point estimate.

2.2 Importance sampling

In importance sampling, the posterior distribution is approximated by drawing samples $\mathbf{z}_i$ from a proposal distribution and assigning a weight to each of them depending on the ratio between the target posterior distribution and the proposal for the respective sample. In our first implementation of importance sampling in this project, the same proposal as for the Metropolis-Hastings algorithm was used (see equations 6-8). However, instead of changing the values of S_0 and $\mathbf{D}$ based on the previous sample as was done in the Metropolis-Hastings algorithm, both S_0 and $\mathbf{D}$ were here kept constant at the value of the point estimate based on the data for all samples. According to Bayes' theorem, we can obtain a value for the unnormalized posterior distribution, here denoted $\pi_0(\mathbf{z}_i)$, by multiplying the prior with the likelihood. In log-space, we hence get the logarithm of the unnormalized posterior

by summing the logarithm of the prior with the logarithm of the likelihood. The weights w_i are then obtained through

$$w_i = \frac{\pi_0(\mathbf{z}_i)}{r(\mathbf{z}_i)} \quad (9)$$

where $r(\mathbf{z}_i)$ is the value of the proposal distribution at $\mathbf{z}_i$. Since we're still working with log-probabilities, the division in equation 9 corresponds to subtraction. Since we hereby get the logarithm of the weights, the logarithm is removed by exponentiating. Finally, the weights are normalized by dividing each weight by the sum of all weights.

Whether importance sampling is an efficient inference method or not is highly dependent on what proposal is used. Therefore, it plays an important role for the efficiency what parameters γ and ν are used in the proposal (equations 7 and 8). To find suitable values for γ and ν , different values for these parameters were tested while tracking the effective sample size $\hat{N}_{ESS}$ defined as

$$\hat{N}_{ESS} = \frac{1}{\sum_i 1^N w_i} \quad (10)$$

where N is the number of samples and w_i are the weights. A high effective sample size close to the actual number of samples used indicates high efficiency, while a low effective sample size indicates that the proposal might have to be adjusted for the importance sampling to work well. Figure 2 shows how the effective sample size varies for different values of γ and ν used in the proposal.

First, the value of ν was fixed at 200 and γ was varied between 10^{-7} and 10^2 . As can be seen in Figure 2a, the effective sample size is low for values of γ above 10 and varies a lot below that. $\gamma = 0.0217$ gave the highest effective sample size and was therefore used to try different values of ν in Figure 2b. Figure 2b shows that the highest effective sample size is obtained for $\nu \approx 200$. Both figures were created using 1000 samples and in both figures, a highest effective sample size of about 120 is obtained. The highest achieved effective sample size is therefore about 12% of the actual sample size.

To try and improve the effective sample size, the proposal from the Metropolis-Hastings algorithm (equations 6-8) was replaced by our Laplace approximation, thus combining it with importance sampling. Since the Laplace approximation should look reasonably similar to our target distribution, this should result in more equal sample weights and hence a higher effective sample size. With this approach, an effective sample size of 2225 was obtained for a run using 10000 samples, corresponding to roughly 22%. This is an increase compared to what was obtained using the proposal from the Metropolis-Hastings algorithm, but still not ideal. Figure 3 shows the resulting distributions for S_0 , mean diffusivity, fractional anisotropy and acute angle when using importance sampling with 10000 samples and the Laplace approximation as the proposal.

2.3 Variational inference

Variational inference (VI) approximates the posterior distribution with a member of a simpler, tractable family of densities by minimizing the Kullback–Leibler (KL) divergence (Blei et al. (2017)), which is equivalent to maximizing the Evidence Lower Bound (ELBO). This approach turns the inference problem into an optimization problem.

In our implementation, we employed a mean-field approximation where the variational posterior is factorized as $q(z) = q(S_0)q(D)$. $q(S_0)$ is modeled as a Gamma distribution, $\Gamma(\alpha, \beta)$, and $q(D)$ is modeled as a Wishart distribution, $W(D, \nu)$ (Blei et al. (2017)). To ensure parameter constraints are respected, the nine unconstrained variational parameters $\theta = (\theta_1, \dots, \theta_9)$ were reparameterized. Specifically, the Gamma shape and scale parameters were defined as $\alpha = e^{\theta_1}$ and $\beta = e^{\theta_2}$. The degrees of freedom for the Wishart distribution, ν , was set such that $\nu = e^{\theta_9} + 2$, ensuring it is at least 2. The scale matrix Σ of the Wishart distribution for D was derived from θ_3 through θ_8 using its Cholesky factorization, $D = LL^\top$. This parameterization ensures that the baseline signal S_0 is positive and the diffusion tensor D remains positive-definite when sampling from $q(z)$.

We fitted this variational posterior by maximizing the ELBO with respect to θ , utilizing a stochastic gradient ascent method with Adam optimizer used to accelerate convergence. The gradient of the ELBO was estimated using the REINFORCE leave-one-out estimator (as shown).

$$\nabla_{\theta} \mathbb{E}_{q^*} \theta[f(z)] = \mathbb{E}_{q^*} \theta[f(z), \nabla_{\theta} \log q_{\theta}(z)] \approx \frac{1}{K} \sum_{k=1}^K \left(f(z_k) - \frac{1}{K-1} \sum_{j \neq k} f(z_j) \right), \nabla_{\theta} \log q_{\theta}(z_k) \quad (11)$$

The provided `variational_posterior` class manages these parameter transformations and contains the necessary method (score) to compute the score $\nabla_{\theta} \log q_{\theta}(z)$ analytically, which is required for the gradient calculation. The optimization was run for a maximum number of iterations (`max_iters=500`), employing $K = 10$ Monte Carlo samples per step to estimate the gradient.

2.4 Laplace approximation

The Laplace approximation provides a deterministic and computationally efficient way to approximate the posterior distribution. The method assumes that the posterior $p(\mathbf{z} | \mathbf{y})$ can be well-approximated by a multivariate normal centered at its mode (the maximum a posteriori, MAP, estimate). The log-posterior is expanded in a second-order Taylor series around the MAP point $\mathbf{z}_{MAP}$:

$$\log p(\mathbf{z} | \mathbf{y}) \approx \log p(\mathbf{z}_{MAP} | \mathbf{y}) - \frac{1}{2}(\mathbf{z} - \mathbf{z}_{MAP})^{\top} H(\mathbf{z} - \mathbf{z}_{MAP}), \quad (12)$$

where H is the Hessian matrix of the negative log-posterior evaluated at $\mathbf{z}_{MAP}$. This yields the Gaussian approximation

$$q(\mathbf{z}) = \mathcal{N}(\mathbf{z}_{MAP}, H^{-1}). \quad (13)$$

In our implementation, we reparameterized the variables as $\mathbf{z} = [\log S_0, \boldsymbol{\theta}]$, where $\boldsymbol{\theta}$ are the six unconstrained parameters obtained from the Cholesky factorization of the diffusion tensor $D = LL^{\top}$. This ensures that $S_0 > 0$ and D remains positive-definite during optimization.

The MAP estimate was found by minimizing the negative log-posterior,

$$-\log p(\mathbf{z} | \mathbf{y}) = -\log p(\mathbf{y} | \mathbf{z}) - \log p(\mathbf{z}), \quad (14)$$

using the BFGS optimization algorithm. The likelihood $p(\mathbf{y} | \mathbf{z})$ was modeled as a Gaussian noise model around the diffusion signal $S_i = S_0 e^{-q_i^{\top} D q_i}$, and the prior $p(\mathbf{z})$ combined gamma distributions for S_0 and the tensor eigenvalues. The optimization converged after approximately 15 iterations, yielding a well-defined MAP point.

To quantify the local curvature of the posterior, we numerically computed the Hessian matrix using finite differences of the negative log-posterior. The covariance of the Laplace approximation was then obtained as the inverse of this Hessian, with a small diagonal regularization term for numerical stability. The posterior is thus approximated as:

$$q(\log S_0, \boldsymbol{\theta}) = \mathcal{N}((\log S_0)_{MAP}, \Sigma_{S_0}), \quad \mathcal{N}(\boldsymbol{\theta}_{MAP}, \Sigma_{\boldsymbol{\theta}}), \quad (15)$$

where Σ_{S_0} and $\Sigma_{\boldsymbol{\theta}}$ denote the corresponding covariance blocks.

The approximation captured the local uncertainty well: the variance of $\log S_0$ was small, indicating that the baseline signal was well constrained, while the determinant of $\Sigma_{\boldsymbol{\theta}}$ was very small, implying tight confidence around the estimated diffusion tensor. Sampling from the Gaussian approximation and transforming back to S_0 and D allowed us to visualize the resulting posterior distributions. Figure 5 shows the estimated distributions for S_0 , mean diffusivity, fractional anisotropy, and acute angle. The distributions are narrow and approximately symmetric, confirming that the posterior is close to Gaussian near its mode, validating the assumptions of the Laplace approximation. The acute angle distributions centered around 1° further indicate that all inference methods produced consistent and near-identical orientation estimates.

3 Comparison

Table 1 shows the results for the four inference methods. Three of the four approaches produced consistent posterior estimates with only minor numerical differences. The estimated baseline signal S_0 was around 335–338 with standard deviations of approximately 8–9, while the mean diffusivity and fractional anisotropy values were also nearly identical across methods. The acute angle distributions were sharply concentrated around 1° , confirming that the estimated principal eigenvectors are

almost perfectly aligned with the reference orientation. These results confirm that the three methods capture the same posterior mode and orientation direction with minimal angular deviation. However, the Variational Inference method failed to converge to the global posterior mode, yielding biased mean estimates for some parameters (e.g., Angle mean $\approx 4.7^\circ$) and overestimated standard deviation estimates. This discrepancy arises from the optimizer becoming trapped in a shallow local optimum of the non-convex ELBO surface, likely worsened by the high variance of the stochastic gradient estimator, as illustrated in Figure 4 (The below results were obtained only after 10,000 `max_iters`). The Laplace approximation achieved these results with a fraction of the computational cost compared to the sampling-based approaches. Importance sampling showed slightly lower precision due to its limited effective sample size, while Metropolis-Hastings required longer runtime to obtain independent samples. Overall, the Laplace approximation provided the most efficient and practically accurate solution, closely matching the stochastic methods while being considerably faster.

Table 1: Comparison of posterior summary statistics for Metropolis-Hastings (MH), Importance Sampling (IS), Variational Inference (VI), and Laplace Approximation (LA) using 10 000 samples.

Metric	MH	IS	VI	LA
S_0 Mean	338.30	335.20	311.86	337.10
S_0 Std	8.63	8.69	29.40	8.91
MD Mean [$\times 10^{-3}$]	0.581	0.574	0.589	0.577
MD Std [$\times 10^{-3}$]	0.020	0.021	0.141	0.021
FA Mean	0.863	0.867	0.907	0.862
FA Std	0.015	0.015	0.036	0.016
Acute Angle Mean [$^\circ$]	1.29	1.30	4.71	1.31
Acute Angle Std [$^\circ$]	0.65	0.67	2.64	0.69

4 Discussion

Metropolis-Hastings is able to approximately reflect the true posterior distributions if we get enough effective samples, and tune the variance parameters to explore the whole uncertainty range. We achieve this with a not too narrow or too wide proposal, to avoid correlated/close samples and low acceptance respectively. So the main strength of Metropolis-Hastings is that it converges to the true posterior. The drawbacks include the slow convergence. The samples are correlated with each other in the Markov chain and can thus represent the wrong distribution until we get enough distance in the chain to consider far samples as independent. Another drawback is the tuning of the proposal parameters, that can also be challenging and time-consuming. As each sample is relatively expensive to compute, and many thousand samples might be needed for each voxel, Metropolis-Hastings wouldn't be preferable to use for full scan voxel scale. As it is a Markov chain, the samples also can't be drawn in parallel. The different voxels could still be parallelized though.

One of the main advantages of importance sampling is that it is an asymptotically exact method, meaning that the approximation approaches the correct posterior distribution for a large number of samples. A disadvantage of importance sampling is that it can be quite computationally expensive compared to deterministic inference methods, especially when the effective sample size is low as in our case. Since, however, the individual draws in importance sampling are independent, the sample generation is easily parallelizable across multiple cores, decreasing the computation time. This puts importance sampling at an advantage, for example compared to the Metropolis-Hastings algorithm in which each sample is dependent on the previous one and the sampling therefore cannot be parallelized. Another thing to keep in mind when using importance sampling is that it can be difficult to find a proposal close enough to the target distribution. An unsuited proposal causes a low effective sample size, which increases the computation time to get an accurate result even more. One way to potentially improve the performance of the importance sampling method in this project could therefore be to try even more different proposals, to see if it is possible to obtain a larger effective sample size and therefore a more accurate solution for the same number of samples. Alternatively, a more accurate solution could of course also be obtained by simply running the method for longer with a larger number of samples.

The Laplace approximation, unlike the stochastic methods above, is deterministic and therefore much faster. Its main strength lies in computational efficiency, as it only requires finding the maximum a posteriori (MAP) estimate and computing the curvature of the posterior around this point. The resulting Gaussian approximation captures the local uncertainty well when the posterior is unimodal and approximately normal, as was the case for our voxel. Another advantage is that the Laplace approximation provides direct access to both the mean and covariance of the approximate posterior, which can be used as input to other methods such as importance sampling. However, the method has clear weaknesses: it cannot capture multimodal or highly skewed posteriors, and its accuracy deteriorates if the true posterior deviates significantly from a Gaussian shape. In such cases, the uncertainty is often underestimated.

Variational inference (VI) offers another deterministic alternative to sampling methods by reframing posterior approximation as an optimization problem, resulting in faster computation times than MCMC methods. VI maximizes the Evidence Lower Bound (ELBO) using a stochastic gradient method, such as the Adam optimizer, utilizing Monte Carlo estimates to approximate the gradient. The main strength of VI is its speed and amenability to large datasets. However, VI introduces specific compromises regarding accuracy: the mean-field assumption dictates that it cannot capture correlations between latent variables, and in our case it overestimates the variance of the true posterior distribution. Furthermore, because the ELBO is typically a non-convex objective function, VI is susceptible to converging only to a local optimum, which is sensitive to initialization.

In terms of scalability for full-brain diffusion MRI analysis involving $\approx 10^5$ voxels, the deterministic and optimization-based methods, namely the Laplace approximation (LA) and Variational Inference (VI), are the most practical solutions among the methods tested. The Laplace approximation excels in efficiency, as it is deterministic and requires only finding the Maximum A Posteriori (MAP) estimate and computing the local curvature around this point. LA is trivially parallelizable across voxels and demonstrated extremely fast convergence (approximately 15 iterations). Similarly, VI turns inference into an optimization problem and is generally faster than stochastic sampling methods. Because both LA and VI calculate the approximation independently per voxel, they are highly parallelizable. In contrast, Metropolis-Hastings (MH) and Importance Sampling (IS) require thousands of samples per voxel to approximate the posterior accurately. Although IS allows for parallel generation of independent samples across cores for a single voxel, MH is sequential and cannot be parallelized within a single Markov chain because each step depends on the previous state. Therefore, while MH and IS remain the most accurate methods for approximating complex posteriors asymptotically, LA and VI offer a better balance between computational cost and approximation quality for large-scale applications.

5 Use of Generative AI

The Gemini autocompletion tool in google colab was used to speed up coding and commenting code. We also used it to generate some initial code snippets which we then treated as a base and significantly adapted and expanded ourselves. We also used it to help us improve the structure and clarity of our report answers, but all content was reviewed and refined by us.

References

Blei, D. M., Kucukelbir, A., and McAuliffe, J. D. (2017). Variational inference: A review for statisticians. *Journal of the American statistical Association*, 112(518):859–877.

A Appendices and Supplementary Material

A.1 Plots and Figures

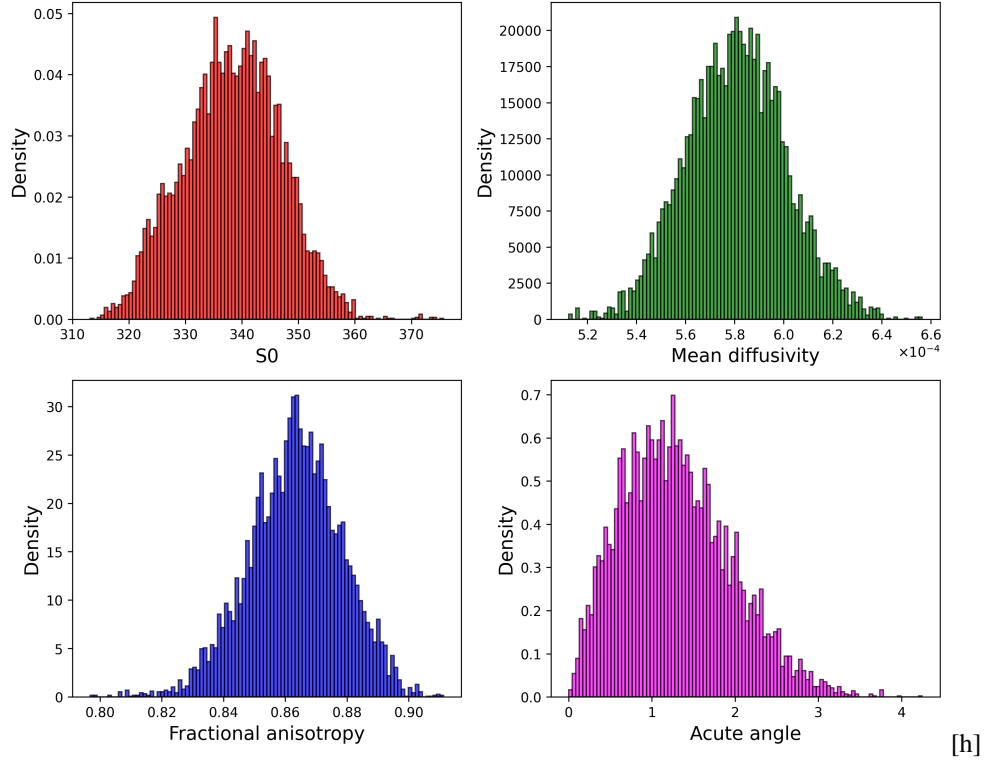


Figure 1: Resulting posterior distribution approximation using the Metropolis-Hastings algorithm and 10 000 samples.

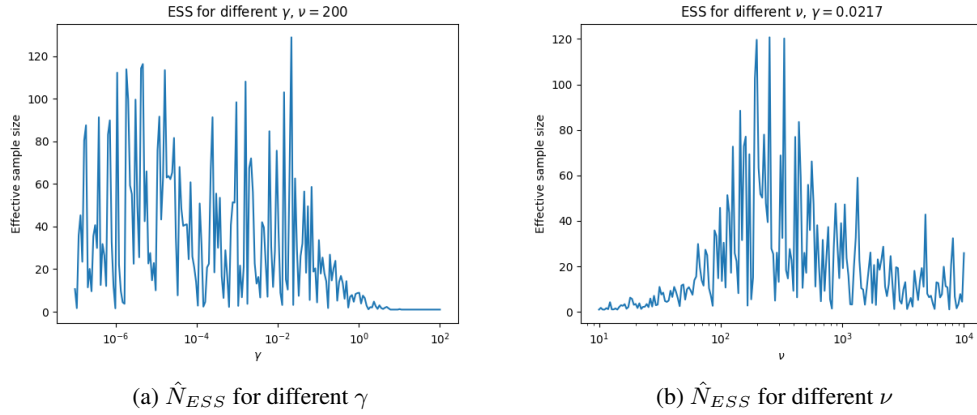


Figure 2: Effective sample size for different values of γ and ν

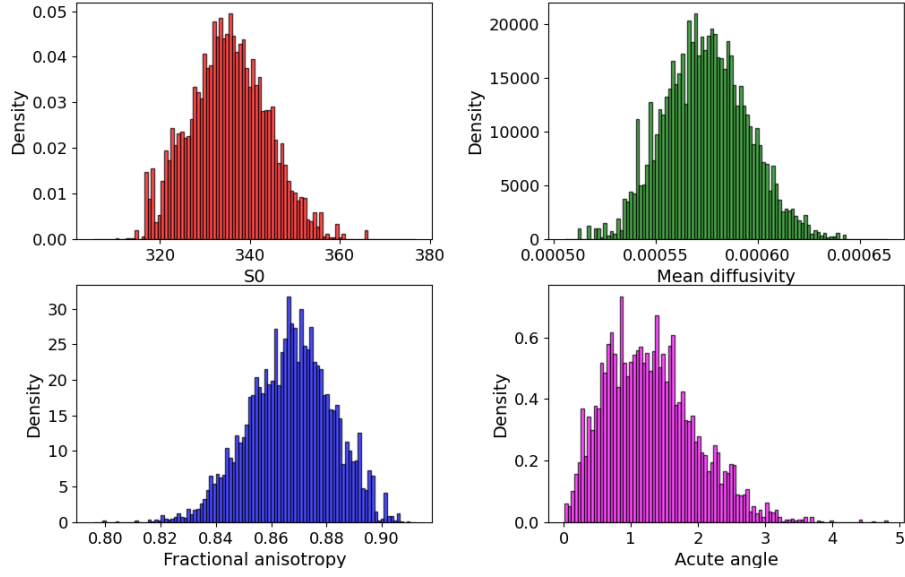


Figure 3: Distributions for S_0 , mean diffusivity, fractional anisotropy and acute angle using importance sampling with Laplace approximation proposal.

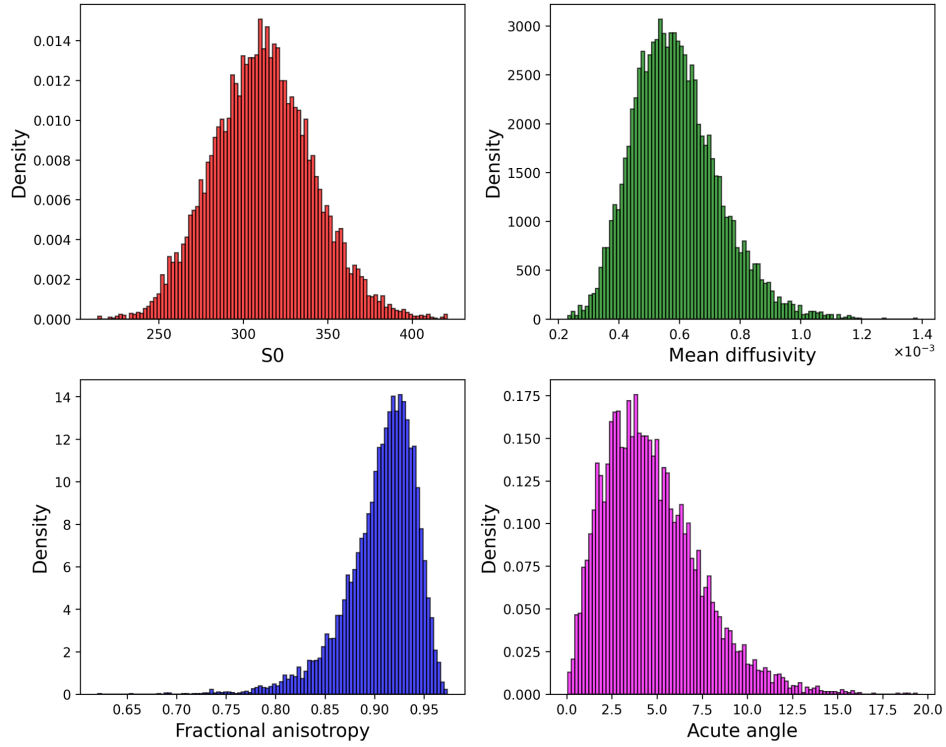


Figure 4: Distributions for S_0 , mean diffusivity, fractional anisotropy, and acute angle obtained using the Variational Inference using ELBO maximization.

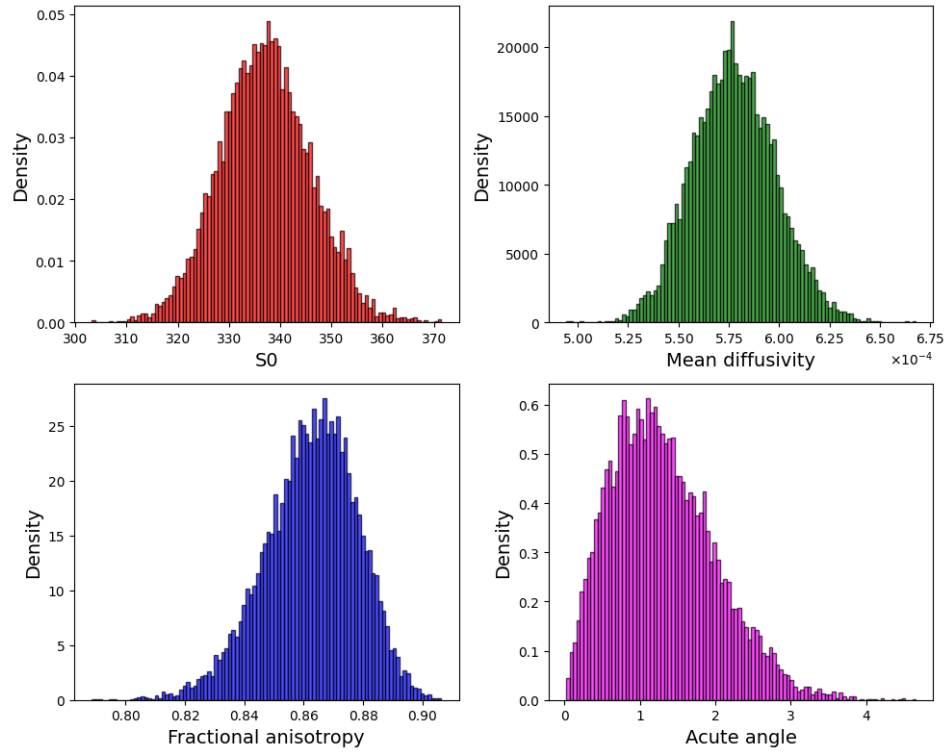


Figure 5: Distributions for S_0 , mean diffusivity, fractional anisotropy, and acute angle obtained using the Laplace approximation with 10 000 samples.

A.2 Code

```
1 # -*- coding: utf-8 -*-
2 """APML_Project.ipynb"""
3
4 # Imports
5
6 #Installs
7 !pip install DIPY
8
9 #IMPORTS
10 # Standard library: general utilities
11 import os
12 import pickle
13 import hashlib
14 from functools import wraps
15 from tqdm import tqdm
16
17 # NumPy and Matplotlib: math and plotting
18 import numpy as np
19 import matplotlib.pyplot as plt
20 import matplotlib.ticker as mticker
21
22 # SciPy: probability distributions, math functions, and optimization
23 from scipy.stats import gamma, norm, wishart, multivariate_normal
24 from scipy.spatial.transform import Rotation
25 from scipy.special import logsumexp, digamma
26 from scipy.optimize import minimize
27
28 # DIPY: diffusion MRI utilities and models
29 from dipy.io.image import load_nifti # for loading / saving imaging
30     ↪ datasets
31 from dipy.io.gradients import read_bvals_bvecs # for loading /
32     ↪ saving our bvals and bvecs
33 from dipy.core.gradients import gradient_table # for constructing
34     ↪ gradient table from bvals/bvecs
35 from dipy.data import get_fnames # for small
36     ↪ datasets that we use in tests and examples
37 from dipy.segment.mask import median_otsu # for masking out
38     ↪ the background
39 import dipy.reconst.dti as dti # for diffusion
40     ↪ tensor model fitting and metrics
41
42 """
43 =====
44 Caching Utility
45 =====
46 Provides disk-based memoization to avoid recomputation.
47 """
48
49 def disk_memoize(cache_dir="cache"):
50     """Decorator for caching function outputs on disk."""
51     def decorator(func):
52         @wraps(func)
53         def wrapper(*args, **kwargs):
54             force = kwargs.pop("force_recompute", False)
55             os.makedirs(cache_dir, exist_ok=True)
56             func_name = func.__name__
57             key = (func_name, args, kwargs)
58             hash_str = hashlib.md5(pickle.dumps(key)).hexdigest()
59             cache_path = os.path.join(cache_dir,
60                                     ↪ f"{func_name}_{hash_str}.pkl")
61
62             if not force and os.path.exists(cache_path):
```

```

56         with open(cache_path, "rb") as f:
57             return pickle.load(f)
58
59         result = func(*args, **kwargs)
60         with open(cache_path, "wb") as f:
61             pickle.dump(result, f)
62
63         return result
64
65     return wrapper
66     return decorator
67
68 """
69 =====
70 Data Loading & Preprocessing
71 =====
72 Loads the Stanford HARDI dataset, applies masking/cropping, and
73 extracts one voxel with a DTI point estimate for testing.
74 """
75
76 @disk_memoize()
77 def get_preprocessed_data():
78     """
79     Load and preprocess a single voxel of diffusion MRI data.
80
81     Returns
82     -----
83     y : ndarray
84         Observed diffusion MRI signal vector for a single voxel.
85     point_estimate : [S0, evals, evecs]
86         Estimated baseline signal, eigenvalues, and eigenvectors.
87     gtab : GradientTable
88         Gradient table with b-values and b-vectors.
89     """
90     data, mask, gtab = get_data()
91     tenmodel = dti.TensorModel(gtab, return_S0_hat=True)
92     y = data[35, 35, 30, :]
93     tenfit = tenmodel.fit(y)
94     S0 = tenfit.S0_hat
95     evals = tenfit.evals
96     evecs = tenfit.evecs
97     point_estimate = [S0, evals, evecs]
98
99     return y, point_estimate, gtab
100
101 def get_data():
102     """
103     Load and preprocess the Stanford HARDI diffusion MRI dataset.
104
105     Returns
106     -----
107     maskdata : ndarray
108         The masked and cropped diffusion MRI data.
109     mask : ndarray (boolean)
110         The brain mask used to exclude background voxels.
111     gtab : GradientTable
112         Gradient information (b-values and b-vectors) for each
113         ↪ measurement.
114     """
115     hardi_fname, hardi_bval_fname, hardi_bvec_fname =
116         ↪ get_fnames('stanford_hardi')
117     data, _ = load_nifti(hardi_fname)
118     bvals, bvecs = read_bvals_bvecs(hardi_bval_fname,
119         ↪ hardi_bvec_fname)
120     gtab = gradient_table(bvals, bvecs)

```

```

118     maskdata, mask = median_otsu(
119         data, vol_idx=range(10, 50), median_radius=3, numpass=1,
120         ↪ autocrop=True, dilate=2
121     )
122
123     print('Loaded data with shape: (%d, %d, %d, %d)' % maskdata.shape)
124
125     return maskdata, mask, gtab
126
127     """
128     =====
129     Linear Algebra Helpers
130     =====
131     Functions for reconstructing tensors and switching between
132     parameterizations.
133     """
134
135     def compute_D(evals, V):
136         """
137         Reconstruct the diffusion tensor D from eigenvalues and
138         ↪ eigenvectors.
139         D = V      V.T
140
141         Parameters
142         -----
143         evals : ndarray
144             Eigenvalues, shape (3,) or batched.
145         V : ndarray
146             Eigenvectors, shape (3, 3) or batched.
147
148         Returns
149         -----
150         D : ndarray
151             Diffusion tensor(s), shape (... , 3, 3).
152         """
153
154         if evals.ndim == 1:
155             evals = evals[None, None, :]
156         elif evals.ndim == 2:
157             evals = evals[:, None, :]
158         if V.ndim == 2:
159             V = V[None, :, :]
160
161         V_scaled = V * evals
162         D = np.matmul(V, np.transpose(V_scaled, axes=[0, 2, 1]))
163
164         return D
165
166     def theta_from_D(D):
167         """
168         Convert a diffusion tensor D into an unconstrained parameter
169         ↪ vector theta.
170         Follows Eq. (18):  $D = L L.T$  with L from Cholesky factorization.
171
172         Parameters
173         -----
174         D : ndarray (3, 3)
175             Symmetric positive-definite diffusion tensor.
176
177         Returns
178         -----
179         theta : ndarray (6,)

```

```

179         Unconstrained parameter vector corresponding to the
180             ↪ lower-triangular
181         entries of L (log of diagonals, raw off-diagonals).
182     """
183     L = np.linalg.cholesky(D)
184     p = D.shape[0]
185     tril_indices = np.tril_indices(p)
186     theta = []
187
188     for i, j in zip(*tril_indices):
189         if i == j:
190             theta.append(np.log(L[i, j]))
191         else:
192             theta.append(L[i, j])
193
194     return np.array(theta)
195
196 def D_from_theta(theta):
197     """
198     Convert unconstrained parameter vector theta back into diffusion
199         ↪ tensor D.
200     Follows Eq. (18):  $D = L L^T$  with L constructed from theta.
201
202     Parameters
203     -----
204     theta : ndarray (...)
205         Unconstrained parameters (log-diagonals, raw off-diagonals).
206
207     Returns
208     -----
209     D : ndarray (...)
210         Symmetric positive-definite diffusion tensor(s).
211     """
212     theta = np.asarray(theta)
213     *batch_shape, _ = theta.shape
214     assert theta.shape[-1] == 6, "Last dimension must be 6 for 3x3
215         ↪ lower-triangular matrices."
216
217     L = np.zeros((*batch_shape, 3, 3), dtype=theta.dtype)
218
219     tril_indices = np.tril_indices(3)
220     for k, (i, j) in enumerate(zip(*tril_indices)):
221         if i == j:
222             L[..., i, j] = np.exp(theta[..., k])
223         else:
224             L[..., i, j] = theta[..., k]
225
226     D = L @ np.swapaxes(L, -1, -2)
227
228     return D.squeeze()
229
230 def grad_D_wrt_theta_at_D(D):
231     """
232     Compute nabla_theta D evaluated at D, using the parameterization
233          $D = L L^T$ .
234
235     Returns
236     -----
237     grad_D : ndarray (3, 3, 6)
238         Gradient of D w.r.t. theta, one 3x3 matrix per parameter.
239     """
240     p = D.shape[0]
241     L = np.linalg.cholesky(D)

```

```

241     tril_indices = np.tril_indices(p)
242     num_params = len(tril_indices[0])
243     grad_D = np.zeros((p, p, num_params))
244
245     for k, (m, n) in enumerate(zip(*tril_indices)):
246         E_mn = np.zeros((p, p))
247         if m == n:
248             factor = L[m, n]
249         else:
250             factor = 1.0
251         E_mn[m, n] = factor
252
253         dD_k = E_mn @ L.T + L @ E_mn.T
254         grad_D[:, :, k] = dD_k
255
256     return grad_D
257
258     """
259     =====
260     Bayesian Model Components
261     =====
262     """
263
264     class frozen_prior:
265         """Prior distribution for (S0, Lam, V)."""
266
267         def __init__(self):
268             self.sigma = 29
269             self.alpha_S = 2
270             self.theta_S = 500
271             self.alpha_Lam = 4
272             self.theta_Lam = 2.5e-4
273
274         def rvs(self):
275             S_0 = gamma.rvs(self.alpha_S, scale= self.theta_S)
276             Lam = gamma.rvs(self.alpha_Lam, scale= self.theta_Lam, size=3)
277             V = Rotation.random().as_matrix()
278             return S_0, Lam, V
279
280         def logpdf(self, S0, lambdas, V):
281             log_p_S0 = gamma.logpdf(S0, self.alpha_S, scale=self.theta_S)
282             log_p_lambda1 = gamma.logpdf(lambdas[0], self.alpha_Lam,
283                 ↪ scale=self.theta_Lam)
284             log_p_lambda2 = gamma.logpdf(lambdas[1], self.alpha_Lam,
285                 ↪ scale=self.theta_Lam)
286             log_p_lambda3 = gamma.logpdf(lambdas[2], self.alpha_Lam,
287                 ↪ scale=self.theta_Lam)
288             log_p_V = 0
289             log_p_z = log_p_S0 + log_p_lambda1 + log_p_lambda2 +
290                 ↪ log_p_lambda3 + log_p_V
291             return log_p_z
292
293     class frozen_likelihood:
294         """Likelihood function for the diffusion tensor model."""
295
296         def __init__(self, gtab, y, sigma=29):
297             """
298             Parameters:
299             - gtab: Gradient table
300             - y: Observed diffusion MRI signal
301             - sigma: Noise standard deviation
302             """
303             self.gtab = gtab
304             self.y = y

```

```

302         self.sigma = sigma
303
304     def logpdf(self, S0, evals, evecs):
305         """
306         Compute log-likelihood of observed signal given tensor
307             ↪ parameters.
308         Assumes Gaussian noise:  $\log p(y|S0,D) = -0.5 * ||y - S||^2 /$ 
309             ↪  $\sigma^2 - \text{const}$ 
310
311         Parameters:
312         - S0: Baseline signal (scalar)
313         - evals: Eigenvalues of diffusion tensor (3,)
314         - evecs: Eigenvectors of diffusion tensor (3,3)
315
316         Returns:
317         - log_likelihood: Log-probability of observed data
318         """
319         S0 = np.atleast_1d(S0)
320         D = compute_D(evals, evecs)
321         q = np.sqrt(self.gtab.bvals[:, None]) * self.gtab.bvecs
322
323         #  $S_i = S0 * \exp(-q_i^T D q_i)$ 
324         S = S0[:, None] * np.exp(-np.einsum('...j, ijk, ...k->i...',
325             ↪ q, D, q))
326         S = S.squeeze()
327
328         residuals = self.y - S
329         log_likelihood = -0.5 * np.sum(residuals**2) / (self.sigma**2)
330
331         n_measurements = len(self.y)
332         log_likelihood -= 0.5 * n_measurements * np.log(2 * np.pi *
333             ↪ self.sigma**2)
334
335         return log_likelihood
336
337     """
338     =====
339     Posterior Approximations
340     =====
341     """
342
343     class variational_posterior:
344         """Variational posterior approximation  $q_{\theta}(S0, D) =$ 
345             ↪  $q(S0)q(D)$ ."""
346
347         def __init__(self, theta):
348             self.shape = np.exp(theta[0])
349             self.scale = np.exp(theta[1])
350             self.df = np.exp(theta[8]) + 2
351             self.Sigma = D_from_theta(theta[2:8])
352
353         def logpdf(self, S0, D):
354             logp_S0 = gamma.logpdf(S0, a=self.shape, scale=self.scale)
355             if D.ndim == 3 and D.shape[0] == 1:
356                 D = D[0]
357             logp_D = wishart.logpdf(D, df=self.df, scale=self.Sigma)
358
359             return logp_S0 + logp_D
360
361         def rvs(self, size):
362             S0_samples = gamma.rvs(a=self.shape, scale=self.scale,
363                 ↪ size=size)
364             D_samples = wishart.rvs(df=self.df, scale=self.Sigma,
365                 ↪ size=size)

```

```

360     evals_samples = []
361     evecs_samples = []
362
363     for D_ in D_samples:
364         evals, evecs = np.linalg.eigh(D_)
365         idx = np.argsort(evals)[::-1]
366         evals_samples.append(evals[idx])
367         evecs_samples.append(evecs[:, idx])
368
369     evals_samples = np.array(evals_samples)
370     evecs_samples = np.array(evecs_samples)
371
372     return S0_samples, evals_samples, evecs_samples
373
374
375 def score(self, S0, D):
376     # Combine score contributions from gamma and Wishart parts
377     score_wrt_log_shape, score_wrt_log_scale =
378         ↪ self.gamma_score(S0)
379     score_wrt_theta, score_wrt_log_df = self.wishart_score(D)
380     return np.concatenate([
381         np.atleast_1d(score_wrt_log_shape),
382         np.atleast_1d(score_wrt_log_scale),
383         np.atleast_1d(score_wrt_theta),
384         np.atleast_1d(score_wrt_log_df)
385     ])
386
387 def gamma_score(self, x):
388     # Score function for gamma distribution
389     score_wrt_log_shape = (np.log(x / self.scale) -
390         ↪ digamma(self.shape)) * self.shape
391     score_wrt_log_scale = (x / self.scale**2 - self.shape /
392         ↪ self.scale) * self.scale
393     return score_wrt_log_shape, score_wrt_log_scale
394
395 def wishart_score(self, D):
396     # Score function for Wishart distribution
397     W = self.df * D
398     Sigma_inv = np.linalg.inv(self.Sigma)
399     score_wrt_Sigma = 0.5 * Sigma_inv @ (W - self.df *
400         ↪ self.Sigma) @ Sigma_inv
401
402     grad_D = grad_D_wrt_theta_at_D(self.Sigma)
403     score_wrt_theta = np.tensordot(
404         score_wrt_Sigma, grad_D, axes=([0, 1], [0, 1])
405     )
406
407     p = W.shape[0]
408     _, logdet_W = np.linalg.slogdet(W)
409     _, logdet_Sigma = np.linalg.slogdet(self.Sigma)
410     digamma_sum = np.sum([digamma((self.df + 1 - j) / 2.0) for j
411         ↪ in range(1, p + 1)])
412     score_wrt_log_df = ((self.df - 2) / 2) * (logdet_W - p *
413         ↪ np.log(2) - logdet_Sigma - digamma_sum)
414     return score_wrt_theta, score_wrt_log_df
415
416 class mvn_reparameterized:
417     """Multivariate Normal approximation for log(S0) and theta (6D D
418         ↪ parameter)."""
419
420     def __init__(self, mean_log_S0, var_log_S0, mean_theta,
421         ↪ cov_theta):
422         """

```

```

417     Parameters:
418     - mean_log_S0: Mean of log(S0)
419     - var_log_S0: Variance of log(S0)
420     - mean_theta: Mean of theta (6D vector parameterizing D)
421     - cov_theta: Covariance matrix of theta (6x6)
422     """
423     self.mean_log_S0 = mean_log_S0
424     self.var_log_S0 = var_log_S0
425     self.mean_theta = mean_theta
426     self.cov_theta = cov_theta
427
428     def rvs(self, size=1):
429         """
430         Sample from the multivariate normal approximation.
431         """
432         log_S0_samples = np.random.normal(self.mean_log_S0,
433             ↪ np.sqrt(self.var_log_S0), size)
434         S0_samples = np.exp(log_S0_samples)
435
436         theta_samples =
437             ↪ np.random.multivariate_normal(self.mean_theta,
438             ↪ self.cov_theta, size)
439
440         D_samples = np.array([D_from_theta(theta) for theta in
441             ↪ theta_samples])
442
443         evals_samples = []
444         evecs_samples = []
445
446         for D in D_samples:
447             evals, evecs = np.linalg.eigh(D)
448             idx = np.argsort(evals)[::-1]
449             evals = evals[idx]
450             evecs = evecs[:, idx]
451
452             evals_samples.append(evals)
453             evecs_samples.append(evecs)
454
455         evals_samples = np.array(evals_samples)
456         evecs_samples = np.array(evecs_samples)
457
458         if size == 1:
459             return S0_samples[0], evals_samples[0], evecs_samples[0]
460         else:
461             return S0_samples, evals_samples, evecs_samples
462
463     def logpdf(self, S0, D):
464         """
465         Evaluate the log-probability of the multivariate normal
466         ↪ approximation.
467         """
468         log_S0 = np.log(S0)
469         logp_S0 = norm.logpdf(log_S0, loc=self.mean_log_S0,
470             ↪ scale=np.sqrt(self.var_log_S0))
471
472         theta = theta_from_D(D)
473         logp_theta = multivariate_normal.logpdf(theta,
474             ↪ mean=self.mean_theta, cov=self.cov_theta)
475
476         return logp_S0 + logp_theta
477
478     """
479     =====
480     Inference Methods

```

```

475 =====
476 """
477
478 @disk_memoize()
479 def metropolis_hastings(n_samples, S0_init, evals_init, evects_init,
480     ↪ gamma_param, nu_param, plot_traces=False):
481     """
482     Metropolis-Hastings algorithm for Bayesian inference.
483     """
484     prior = frozen_prior()
485     y, point_estimate, gtab = get_preprocessed_data()
486     likelihood = frozen_likelihood(gtab, y)
487     S0_samples = np.zeros(n_samples)
488     evals_samples = np.zeros((n_samples,3))
489     evects_samples = np.zeros((n_samples, 3, 3))
490
491     S0_samples[0] = S0_init
492     evals_samples[0] = evals_init
493     evects_samples[0] = evects_init
494
495     def proposal_sample(S0, evals, evects, gamma_param, nu_param):
496         """Computes a sample from the proposal distribution (Gamma for
497             ↪ S0, Wishart for D)."""
498         S0_new = gamma.rvs(np.power(gamma_param,-2),
499             ↪ scale=(np.power(gamma_param,2)*S0))
500         D_prev = compute_D(evals, evects).squeeze()
501         D_new = wishart.rvs(df=nu_param, scale=D_prev/nu_param)
502         evals_new, evects_new = np.linalg.eigh(D_new)
503         idx = np.argsort(evals_new)[::-1]
504         evals_new = evals_new[idx]
505         evects_new = evects_new[:, idx]
506
507         return S0_new, evals_new, evects_new
508
509     def proposal_pdf(S0_new, evals_new, evects_new, S0_prev, evals_prev,
510         ↪ evects_prev, gamma_param, nu_param):
511         """Compute the log probability density of the proposal."""
512         scale_S0_prev = (gamma_param**2) * S0_prev
513         log_q_S0_new_given_prev = gamma.logpdf(S0_new, a=1.0 /
514             ↪ (gamma_param**2), scale=scale_S0_prev)
515
516         D_prev = compute_D(evals_prev, evects_prev).squeeze()
517         D_new = compute_D(evals_new, evects_new).squeeze()
518         log_q_D_new_given_prev = wishart.logpdf(D_new, df=nu_param,
519             ↪ scale=D_prev/nu_param)
520
521         log_q_new_given_prev = log_q_S0_new_given_prev +
522             ↪ log_q_D_new_given_prev
523
524         return log_q_new_given_prev
525
526     accepted=0
527     for i in range(n_samples-1):
528         S0_prev, evals_prev, evects_prev = S0_samples[i],
529             ↪ evals_samples[i], evects_samples[i]
530         S0_new, evals_new, evects_new = proposal_sample(S0_prev,
531             ↪ evals_prev, evects_prev, gamma_param, nu_param)
532
533         # Calculate acceptance ratio in log space
534         log_prior_new = prior.logpdf(S0_new, evals_new, evects_new)
535         log_prior_prev = prior.logpdf(S0_prev, evals_prev, evects_prev)
536
537         log_q_new_given_prev = proposal_pdf(S0_new, evals_new, evects_new,
538             ↪ S0_prev, evals_prev, evects_prev, gamma_param, nu_param)

```



```

591     if use_laplace_proposal:
592         # Laplace approximation as proposal q
593         S0_samples[i], evals_samples[i], evecs_samples[i] =
594             ↪ gauss_approx.rvs()
595         D = compute_D(evals_samples[i], evecs_samples[i]).squeeze()
596         log_r = gauss_approx.logpdf(S0_samples[i], D)
597     else:
598         # Simple proposal q: Gamma for S0, Wishart for D
599         S0_samples[i] = gamma.rvs(gamma_param**(-2),
600             ↪ scale=gamma_param**2*S0_estimate)
601         D = wishart.rvs(nu_param, D_estimate/nu_param)
602         evals_samples[i], evecs_samples[i] = np.linalg.eigh(D)
603         log_r_S0 = gamma.logpdf(S0_samples[i], gamma_param**(-2),
604             ↪ scale=gamma_param**2*S0_estimate)
605         log_r_D = wishart.logpdf(D, nu_param, D_estimate/nu_param)
606         log_r = log_r_S0 + log_r_D
607
608     idx = np.argsort(evals_samples[i])[:, -1]
609     evals_samples[i] = evals_samples[i][idx]
610     evecs_samples[i] = evecs_samples[i][:, idx]
611
612     # Log posterior (target pi_0)
613     log_pi_0 = prior.logpdf(S0_samples[i], evals_samples[i],
614         ↪ evecs_samples[i]) + likelihood.logpdf(S0_samples[i],
615         ↪ evals_samples[i], evecs_samples[i])
616     log_weights[i] = log_pi_0 - log_r
617
618     log_weights -= np.max(log_weights)
619     importance_weights = np.exp(log_weights)
620     weight_sum = np.sum(importance_weights)
621     importance_weights = np.array([w/weight_sum for w in
622         ↪ importance_weights])
623     ess = 1/np.sum(importance_weights**2)
624
625     return importance_weights, S0_samples, evals_samples,
626         ↪ evecs_samples, ess
627
628 @disk_memoize()
629 def variational_inference(max_iters=10000, K=256, learning_rate=1e-4,
630     ↪ theta_init=None):
631     """
632     Variational Inference for fitting the variational posterior
633     ↪ q_theta(z).
634     """
635     y, point_estimate, gtab = get_preprocessed_data()
636     prior = frozen_prior()
637     likelihood = frozen_likelihood(gtab=gtab, y=y)
638
639     # Initialize theta
640     if theta_init is None:
641         S0_init = np.mean(point_estimate[0])
642         D_init = compute_D(point_estimate[1],
643             ↪ point_estimate[2]).squeeze()
644
645         theta_init = np.zeros(9)
646         alpha = 100
647         beta = S0_init/alpha
648
649         theta_init[0] = np.log(alpha)
650         theta_init[1] = np.log(beta)
651         theta_init[8] = np.log(13)
652         df = np.exp(theta_init[8]) + 2
653         theta_init[2:8] = theta_from_D(D_init/df)
654
655     theta = theta_init.copy()

```

```

646
647 # Adam optimizer parameters
648 m = np.zeros_like(theta)
649 v = np.zeros_like(theta)
650 beta1, beta2 = 0.9, 0.999
651 eps = 1e-8
652
653 vp = variational_posterior(theta)
654
655 for t in tqdm(range(1, max_iters + 1)):
656     # Draw K samples from q_theta(z)
657     S0_samples, evals_samples, evecs_samples = vp.rvs(size=K)
658     D_samples = compute_D(evals_samples, evecs_samples)
659
660     # Compute f(z) = log p(D, z) - log q_theta(z)
661     f_z = np.array([prior.logpdf(S0, evals, evecs) +
662                     likelihood.logpdf(S0, evals, evecs) -
663                     vp.logpdf(S0, D)
664                     for S0, D, evals, evecs in zip(S0_samples,
665                                                     ↪ D_samples, evals_samples,
666                                                     ↪ evecs_samples)])
667
668     # Leave-one-out gradient estimator
669     grads = []
670     for k in range(K):
671         f_loo = f_z[k] - np.mean(np.delete(f_z, k))
672         grad_logq = vp.score(S0_samples[k], D_samples[k])
673         grads.append(f_loo * grad_logq)
674     grad_estimate = np.mean(grads, axis=0)
675
676     # Adam update
677     m = beta1 * m + (1 - beta1) * grad_estimate
678     v = beta2 * v + (1 - beta2) * (grad_estimate ** 2)
679     m_hat = m / (1 - beta1 ** t)
680     v_hat = v / (1 - beta2 ** t)
681     theta += learning_rate * m_hat / (np.sqrt(v_hat) + eps)
682
683     vp = variational_posterior(theta)
684
685 return vp
686
687 @disk_memoize()
688 def laplace_approximation():
689     """
690     Laplace approximation: find MAP estimate and approximate posterior
691     with multivariate normal distribution.
692
693     Returns:
694     - posterior: mvn_reparameterized object representing the
695       ↪ approximation
696     """
697     y, point_estimate, gtab = get_preprocessed_data()
698     S0_init, evals_init, evecs_init = point_estimate
699     D_init = compute_D(evals_init, evecs_init).squeeze()
700
701     prior = frozen_prior()
702     likelihood = frozen_likelihood(gtab, y)
703
704     def neg_log_posterior(params):
705         """Negative log-posterior for optimization (params = [log_S0,
706             ↪ theta])."""
707         log_S0 = params[0]
708         theta = params[1:7]
709         S0 = np.exp(log_S0)
710         D = D_from_theta(theta)

```

```

707     evals, evecs = np.linalg.eigh(D)
708     evals = np.maximum(evals, 1e-8)
709     idx = np.argsort(evals)[::-1]
710     evals = evals[idx]
711     evecs = evecs[:, idx]
712
713
714     try:
715         log_prior = prior.logpdf(S0, evals, evecs)
716         log_like = likelihood.logpdf(S0, evals, evecs)
717         log_posterior = log_prior + log_like
718         return -log_posterior
719     except:
720         return 1e10
721
722 log_S0_init = np.log(S0_init)
723 theta_init = theta_from_D(D_init)
724 params_init = np.concatenate([[log_S0_init], theta_init])
725
726 print("==== Starting Laplace approximation optimization
727 ↪ =====")
728 print(f"Initial parameters: log_S0={log_S0_init:.3f}, theta
729 ↪ shape={theta_init.shape}")
730
731 result = minimize(
732     neg_log_posterior,
733     params_init,
734     method='BFGS',
735     options={'disp': True, 'maxiter': 1000}
736 )
737
738 if not result.success:
739     print("Warning: Optimization did not converge!")
740     print(f"Message: {result.message}")
741
742 map_params = result.x
743 map_log_S0 = map_params[0]
744 map_theta = map_params[1:7]
745
746 print(f"MAP estimate found: log_S0={map_log_S0:.3f}")
747 print(f"MAP negative log-posterior: {result.fun:.3f}")
748
749 def hessian_numerical(params, eps=1e-5):
750     """Compute Hessian matrix numerically using finite
751     ↪ differences."""
752     n = len(params)
753     hessian = np.zeros((n, n))
754
755     for i in range(n):
756         for j in range(n):
757             if i <= j:
758                 e_i = np.zeros(n); e_i[i] = eps
759                 e_j = np.zeros(n); e_j[j] = eps
760
761                 if i == j:
762                     hessian[i, j] = (neg_log_posterior(params +
763                     ↪ e_i) -
764                                     2*neg_log_posterior(params) +
765                                     neg_log_posterior(params -
766                                     ↪ e_i)) / (eps**2)
767             else:
768                 hessian[i, j] = (neg_log_posterior(params +
769                 ↪ e_i + e_j) -
770                                     neg_log_posterior(params + e_i
771                                     ↪ - e_j) -

```

```

765         neg_log_posterior(params - e_i
766                             ↪ + e_j) +
767         neg_log_posterior(params - e_i
768                             ↪ - e_j)) / (4*eps**2)
769
770     hessian[j, i] = hessian[i, j]
771
772     return hessian
773
774     print("Computing Hessian at MAP estimate...")
775     hessian = hessian_numerical(map_params)
776
777     try:
778         cov_matrix = np.linalg.inv(hessian + 1e-6 *
779                                     ↪ np.eye(len(hessian)))
780     except np.linalg.LinAlgError:
781         print("Warning: Hessian is singular, using pseudo-inverse")
782         cov_matrix = np.linalg.pinv(hessian)
783
784     mean_log_S0 = map_log_S0
785     var_log_S0 = cov_matrix[0, 0]
786     mean_theta = map_theta
787     cov_theta = cov_matrix[1:7, 1:7]
788
789     print(f"Posterior approximation complete!")
790     print(f"log(S0): mean={mean_log_S0:.3f},
791           ↪ std={np.sqrt(var_log_S0):.3f}")
792     print(f"theta covariance determinant:
793           ↪ {np.linalg.det(cov_theta):.2e}")
794
795     return mvn_reparameterized(mean_log_S0, var_log_S0, mean_theta,
796                               ↪ cov_theta)
797
798 def plot_results(S0, evals, evecs, evec_ref, weights=None, method=""):
799     """
800     Plot posterior results as histograms and save to file.
801
802     Creates histograms of baseline signal (S0), mean diffusivity (MD),
803     fractional anisotropy (FA), and the angle between estimated and
804     reference eigenvectors.
805     """
806     if weights is None:
807         weights = np.ones_like(S0)
808         weights /= np.sum(weights)
809
810     n_bins = np.floor(np.sqrt(len(weights))).astype(int)
811
812     weights = weights.squeeze()
813     S0 = S0.squeeze()
814     md = dti.mean_diffusivity(evals).squeeze()
815     fa = dti.fractional_anisotropy(evals).squeeze()
816
817     angle = 360/(2*np.pi) * np.arccos(np.abs(np.dot(evecs[:, :, 0],
818                                                         ↪ evec_ref)))
819
820     fig, axes = plt.subplots(2, 2, figsize=(10, 8), sharey=False)
821
822     axes[0, 0].hist(S0, bins=n_bins, density=True, weights=weights,
823                     alpha=0.7, color='red', edgecolor='black')
824     axes[0, 0].set_xlabel("S0", fontsize=14)
825     axes[0, 0].set_ylabel("Density", fontsize=14)
826
827     axes[0, 1].hist(md, bins=n_bins, density=True, weights=weights,
828                     alpha=0.7, color='green', edgecolor='black')
829     axes[0, 1].set_xlabel("Mean diffusivity", fontsize=14)

```

```

823 axes[0, 1].set_ylabel("Density", fontsize =14)
824 axes[0,
    ↪ 1].xaxis.set_major_formatter(mticker.ScalarFormatter(useMathText=True))
825 axes[0, 1].xaxis.get_major_formatter().set_scientific(True)
826 axes[0, 1].xaxis.get_major_formatter().set_powerlimits((0, 0))
827
828 axes[1, 0].hist(fa, bins=n_bins, density=True, weights=weights,
829               alpha=0.7, color='blue', edgecolor='black')
830 axes[1, 0].set_xlabel("Fractional anisotropy", fontsize =14)
831 axes[1, 0].set_ylabel("Density", fontsize =14)
832
833 axes[1, 1].hist(angle, bins=n_bins, density=True, weights=weights,
834               alpha=0.7, color='magenta', edgecolor='black')
835 axes[1, 1].set_xlabel("Acute angle", fontsize =14)
836 axes[1, 1].set_ylabel("Density", fontsize =14)
837
838 plt.tight_layout()
839 plt.savefig("results_{}.png".format(method), dpi=300,
    ↪ bbox_inches='tight')
840 plt.show()
841
842 # =====
843 # Initialization and Data Loading
844 # =====
845
846 y, point_estimate, gtab = get_preprocessed_data(force_recompute=False)
847 S0_init, evals_init, evecs_init = point_estimate
848 D_init = compute_D(evals_init, evecs_init).squeeze()
849
850 prior = frozen_prior()
851 likelihood = frozen_likelihood(gtab, y)
852
853 evec_principal = evecs_init[:, 0]
854
855 print("=" * 80)
856 print("INITIALIZATION VALUES (Point Estimate from DTI)")
857 print("=" * 80)
858 print(f"S0_init: {S0_init:.4f}")
859 print(f"evals_init: [{evals_init[0]:.6f}, {evals_init[1]:.6f},
    ↪ {evals_init[2]:.6f}]")
860 print(f"MD_init: {dti.mean_diffusivity(evals_init):.6f}")
861 print(f"FA_init:
    ↪ {dti.fractional_anisotropy(evals_init):.6f}")
862 print(f"D_init trace: {np.trace(D_init):.6f}")
863 print(f"Principal evec: [{evec_principal[0]:.4f},
    ↪ {evec_principal[1]:.4f}, {evec_principal[2]:.4f}]")
864 print(f"\nLog prior: {prior.logpdf(S0_init, evals_init,
    ↪ evecs_init):.4f}")
865 print(f"Log likelihood: {likelihood.logpdf(S0_init, evals_init,
    ↪ evecs_init):.4f}")
866 print(f"Log posterior: {prior.logpdf(S0_init, evals_init,
    ↪ evecs_init) + likelihood.logpdf(S0_init, evals_init,
    ↪ evecs_init):.4f}")
867
868 np.random.seed(0)
869 n_samples = 10000
870
871 # =====
872 # Metropolis Hastings
873 # =====
874
875 MH = True
876 if MH:
877     gamma_param = 1e-2
878     nu_param = 2000

```

```

879 S0_mh, evals_mh, evecs_mh = metropolis_hastings(
880     n_samples, S0_init, evals_init, evecs_init,
881     gamma_param, nu_param,
882     plot_traces=True, force_recompute=False
883 )
884
885 burn_in = 0
886
887
888 print("=" * 80)
889 print("METROPOLIS-HASTINGS RESULTS")
890 print("=" * 80)
891 print(f"Total samples:      {len(S0_mh)}")
892 print(f"Burn-in:             {burn_in}")
893
894 md_mh = dti.mean_diffusivity(evals_mh[burn_in:])
895 fa_mh = dti.fractional_anisotropy(evals_mh[burn_in:])
896 # angle_mh = 360/(2*np.pi) *
897     ↪ np.arccos(np.abs(np.dot(evecs_mh[burn_in:, :, 2],
898     ↪ evec_principal)))
899 dotvals = np.einsum('ij,j->i', evecs_mh[burn_in:, :, 0],
900     ↪ evec_principal) # safer dot per-row
901 dotvals = np.clip(np.abs(dotvals), -1.0, 1.0) # numerical safety
902 angle_mh = np.degrees(np.arccos(dotvals))
903
904 print(f"\nMean Diffusivity Mean: {np.mean(md_mh):.6f}, FA Mean:
905     ↪ {np.mean(fa_mh):.6f}")
906 print(f"Angle Mean: {np.mean(angle_mh):.2f} ")
907
908 plot_results(S0_mh[burn_in:], evals_mh[burn_in:],
909     ↪ evecs_mh[burn_in:, :, :], evec_principal, method="mh")
910
911 # =====
912 # Importance Sampling
913 # =====
914
915 IS = True
916 if IS:
917     gamma_param = 0.02
918     nu_param = 200
919
920     w_is, S0_is, evals_is, evecs_is, ess = importance_sampling(
921         n_samples, gamma_param, nu_param,
922         use_laplace_proposal=True,
923         force_recompute=True
924     )
925
926     w_norm = w_is / np.sum(w_is)
927     md_is = dti.mean_diffusivity(evals_is)
928     fa_is = dti.fractional_anisotropy(evals_is)
929     # angle_is = 360/(2*np.pi) * np.arccos(np.abs(np.dot(evecs_is[:,
930     ↪ :, 2], evec_principal)))
931     dotvals_is = np.einsum('ij,j->i', evecs_is[:, :, 0],
932     ↪ evec_principal)
933     dotvals_is = np.clip(np.abs(dotvals_is), -1.0, 1.0)
934     angle_is = np.degrees(np.arccos(dotvals_is))
935
936     print("=" * 80)
937     print("IMPORTANCE SAMPLING RESULTS")
938     print("=" * 80)
939     print(f"Effective sample size: {ess:.1f}")
940     print(f"S0 mean: {np.sum(w_norm * S0_is):.4f}, MD mean:
941         ↪ {np.sum(w_norm * md_is):.6f}, FA mean: {np.sum(w_norm *
942         ↪ fa_is):.6f}")

```

```

935     plot_results(S0_is, evals_is, evecs_is, evec_principal,
936                 ↪ weights=w_is, method="is")
937
938 # =====
939 # Variational Inference
940 # =====
941
942 VI = True
943 if VI:
944     posterior_vi = variational_inference(force_recompute=True)
945     S0_vi, evals_vi, evecs_vi = posterior_vi.rvs(size=n_samples)
946
947     md_vi = dti.mean_diffusivity(evals_vi)
948     fa_vi = dti.fractional_anisotropy(evals_vi)
949
950     dotvals = np.einsum('ij,j->i', evecs_vi[:, :, 0], evec_principal)
951     dotvals = np.clip(np.abs(dotvals), -1.0, 1.0)
952     angle_vi = np.degrees(np.arccos(dotvals))
953
954     stats = {
955         "S0": (np.mean(S0_vi), np.std(S0_vi)),
956         "MD": (np.mean(md_vi), np.std(md_vi)),
957         "FA": (np.mean(fa_vi), np.std(fa_vi)),
958         "Angle": (np.mean(angle_vi), np.std(angle_vi))
959     }
960
961     print("=" * 80)
962     print("VARIATIONAL INFERENCE RESULTS")
963     print("=" * 80)
964     for key, (mean_val, std_val) in stats.items():
965         print(f"{key} mean: {mean_val:.6f}, {key} std: {std_val:.6f}")
966
967     plot_results(S0_vi, evals_vi, evecs_vi, evec_principal,
968                 ↪ method="vi")
969
970 # =====
971 # Laplace Approximation
972 # =====
973
974 LA = True
975 if LA:
976     posterior_laplace = laplace_approximation(force_recompute=False)
977     S0_laplace, evals_laplace, evecs_laplace =
978         ↪ posterior_laplace.rvs(size=n_samples)
979
980     md_laplace = dti.mean_diffusivity(evals_laplace)
981     fa_laplace = dti.fractional_anisotropy(evals_laplace)
982     # angle_laplace = 360/(2*np.pi) *
983         ↪ np.arccos(np.abs(np.dot(evecs_laplace[:, :, 2],
984         ↪ evec_principal)))
985     dotvals_lap = np.einsum('ij,j->i', evecs_laplace[:, :, 0],
986         ↪ evec_principal)
987     dotvals_lap = np.clip(np.abs(dotvals_lap), -1.0, 1.0)
988     angle_laplace = np.degrees(np.arccos(dotvals_lap))
989
990     print("=" * 80)
991     print("LAPLACE APPROXIMATION RESULTS")
992     print("=" * 80)
993     print(f"S0 mean: {np.mean(S0_laplace):.4f}, MD mean:
994         ↪ {np.mean(md_laplace):.6f}, FA mean:
995         ↪ {np.mean(fa_laplace):.6f}, Angle mean:
996         ↪ {np.mean(angle_laplace):.2f} ")
997
998     plot_results(S0_laplace, evals_laplace, evecs_laplace,
999                 ↪ evec_principal, method="laplace")

```

```
990  
991 print("=" * 80)  
992 print("All experiments completed.")  
993 print("=" * 80)
```